[LOGOTIPO UV / NOMBRE DEL EQUIPO]

Reporte de Práctica de Laboratorio — Programación Orientada a Objetos

EE: Temas Selectos de IIE II (Código: IEDI 18005)

Facultad de Instrumentación Electrónica — Universidad Veracruzana

Información General del Equipo

Número de práctica	Práctica 4
Título de la práctica	Proyecto Integrador: Sistema de control de acceso
Fecha de entrega	16/10/2026
Repositorio de GitHub	https://github.com/DarkZodiac0/mirepositorio_poo/tree/main/proyecto-integrador-ESP32

Nombre completo del integrante	Matrícula UV	Usuario de GitHub
Axel Arturo Pérez Montero	S23013994	@DarkZodiac0
Isaac Gabriel Bremont Limon	S23013923	@Isaaclimon14

Sección 1: Mapeo de Abstracción (Hardware a POO)

El sistema de control de acceso usa un microcontrolador **ESP32** como controlador central. Lee dos sensores (módulo RFID RC522 y sensor de huella digital) y comanda cinco elementos de salida: dos servomotores, una pantalla LCD 16x2 y dos LED. Cada componente físico se representa como un objeto lógico mediante cuatro capas de abstracción:

Capa	Qué contiene	Ejemplo con el servo
1. Física	El dispositivo y sus señales eléctricas	Servomotor, señal PWM de 50 Hz, alimentación de 5 V
2. Interfaz	El bus o protocolo con que se conecta al ESP32	Pin GPIO 13 generando PWM
3. Librería	Código del fabricante o de la comunidad que maneja esa interfaz	`ESP32Servo`: `attach()`, `write(grados)`
4. Objeto lógico	La clase del proyecto, que expone solo lo esencial	`ActuadorServo`: `abrir()` y `cerrar()`

El programa principal trabaja únicamente en la capa 4. No necesita saber qué es un pulso PWM ni qué pin se usa: solo pide `servoEntrada.abrir()`.

El **mapeo de abstracción** consiste en definir, para cada componente, qué propiedades físicas pasan a ser **atributos**, qué operaciones sobre el hardware pasan a ser **métodos** y qué detalles quedan **ocultos** dentro de la clase (encapsulamiento). Con esto se cumple el requisito de al menos dos sensores (RFID y huella) y al menos dos actuadores o indicadores (servos, LCD y LED).

1.1. Diagrama de Clases UML

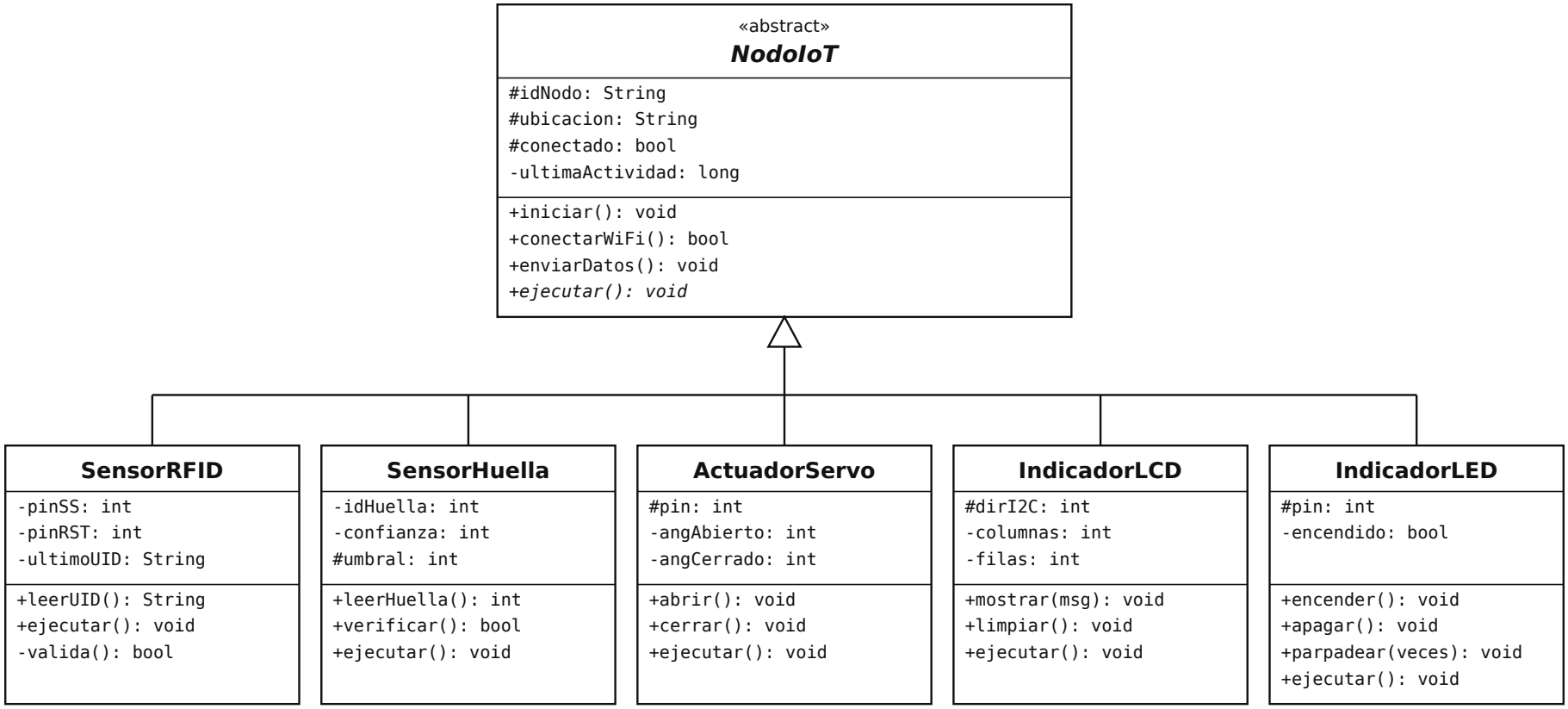


*Figura 1. Diagrama de clases. Las visibilidades se indican con **+** público, **#** protegido y **-** privado. La flecha con triángulo hueco representa la herencia: las cinco clases hijas derivan de la superclase abstracta* `NodoIoT`*, cuyo método abstracto* `ejecutar()` *(en cursiva) cada una implementa a su manera.*

1.2. Mapeo de Atributos (Características Físicas a Variables de Clase)

NodoIoT

- `idNodo` (protegido): identificador único del nodo dentro del sistema (por ejemplo, RFID-01); distingue entre dos componentes iguales.
- `ubicacion` (protegido): lugar físico donde está instalado el componente (puerta principal, entrada, salida).
- `conectado` (protegido): refleja si el ESP32 tiene conexión WiFi activa para enviar datos.
- `ultimaActividad` (privado): marca de tiempo en milisegundos de la última operación del nodo.

SensorRFID

- `pinSS` (privado): pin del ESP32 (GPIO 5) conectado al pin SS (SDA) del RC522, que selecciona el módulo en el bus SPI.
- `pinRST` (privado): pin del ESP32 (GPIO 27) conectado al pin RST del RC522, usado para reiniciar el lector.
- `ultimoUID` (privado): identificador único de la última tarjeta leída, convertido de bytes a texto hexadecimal; es la llave para buscar a la persona en la base de datos.

SensorHuella

- `idHuella` (privado): número de la plantilla de huella que el módulo encontró en su memoria interna (-1 si no hubo coincidencia).
- `confianza` (privado): puntaje de coincidencia que devuelve el módulo biométrico.
- `umbral` (protegido): confianza mínima aceptable; define qué tan estricta es la autenticación.

ActuadorServo

- `pin` (protegido): pin PWM del ESP32 conectado a la señal del servo (GPIO 13 y 14 para los dos servos).
- `angAbierto` (privado): ángulo que corresponde a la posición mecánica de abierto.
- `angCerrado` (privado): ángulo que corresponde a la posición mecánica de cerrado.

IndicadorLCD

- `dirI2C` (protegido): dirección del adaptador I2C de la pantalla en el bus (típicamente 0x27).
- `columnas` y `filas` (privados): dimensiones físicas de la pantalla (16 por 2), que limitan cuánto texto cabe.

IndicadorLED

- `pin` (protegido): salida digital del ESP32 conectada al LED (GPIO 25 verde, GPIO 26 rojo).
- `encendido` (privado): estado lógico actual del LED, para que el objeto sepa si está prendido o apagado.

1.3. Mapeo de Métodos (Acciones del Hardware a Funciones Miembro)

NodoIoT

- `iniciar()`: registra el arranque del nodo; cada subclase lo especializa para configurar su propio hardware (SPI, UART, I2C, PWM o pin digital).
- `conectarWiFi()`: conecta el ESP32 a la red y actualiza el atributo `conectado`.
- `enviarDatos()`: reporta la actividad del nodo al servidor del dashboard.
- `ejecutar()` (abstracto): define la acción principal del nodo; cada subclase lo implementa según su hardware.

SensorRFID

- `leerUID()`: detecta una tarjeta presente, lee su UID por SPI y lo devuelve como texto.
- `valida()` (privado): comprueba que la lectura no esté vacía antes de entregarla.
- `ejecutar()`: dispara una lectura de tarjeta.

`SensorHuella`

- `leerHuella()`: ordena al módulo capturar la imagen, convertirla y buscarla por UART; devuelve el ID encontrado o -1.
- `verificar()`: compara la confianza obtenida contra el `umbral` y decide si la huella es aceptada.
- `ejecutar()`: dispara una lectura de huella.

`ActuadorServo`

- `abrir()` y `cerrar()`: envían al servo el ángulo correspondiente mediante una señal PWM.
- `ejecutar()`: realiza el ciclo de acceso completo (abrir, esperar unos segundos y cerrar).

`IndicadorLCD`

- `mostrar(msg)`: escribe el mensaje en la pantalla a través del bus I2C; un salto de línea separa las dos filas.
- `limpiar()`: borra el contenido de la pantalla.
- `ejecutar()`: muestra el mensaje de espera de tarjeta.

`IndicadorLED`

- `encender()` y `apagar()`: ponen el pin en nivel alto o bajo y actualizan `encendido`.
- `parpadear(veces)`: alterna el LED la cantidad de veces indicada.
- `ejecutar()`: genera un parpadeo como señal visual.

1.4. Conexiones al ESP32

Componente	Interfaz	Pines del ESP32
RC522	SPI	SS: GPIO 5, SCK: 18, MISO: 19, MOSI: 23, RST: 27
Sensor de huella	UART	RX2: GPIO 16 (recibe el TX del sensor), TX2: GPIO 17 (envía al RX del sensor)
LCD 16x2 con adaptador	I2C	SDA: GPIO 21, SCL: GPIO 22
Servo de entrada	PWM	GPIO 13
Servo de salida	PWM	GPIO 14
LED verde	GPIO	GPIO 25 (con resistencia en serie)
LED rojo	GPIO	GPIO 26 (con resistencia en serie)

Todos los componentes comparten tierra (GND). Los servos se alimentan con una fuente de 5 V independiente del ESP32 para evitar reinicios por sus picos de corriente.

Sección 2: El Registro de Errores (Bug Log)

#	Falla física o lógica detectada	Causa raíz (hardware / software)	Solución técnica implementada en código / conexión
1	*[Describir falla real de laboratorio]*	*[Describir causa analizada]*	*[Explicar solución técnica implementada]*
2	*[Describir falla real de laboratorio]*	*[Describir causa analizada]*	*[Explicar solución técnica implementada]*

Sección 3: Justificación del Diseño Orientado a Objetos

1. Herencia y reutilización de código

Sí se utilizó herencia. La **superclase** es `NodoIoT` (abstracta) y las **subclases** son `SensorRFID`, `SensorHuella`, `ActuadorServo`, `IndicadorLCD` e `IndicadorLED`. Lo que todos los componentes comparten (`idNodo`, `ubicacion`, `conectado`, `iniciar()`, `conectarWiFi()` y `enviarDatos()`) se escribe una sola vez en la superclase y se reutiliza en las cinco hijas. Además, hay reutilización por instanciación: los dos servos son dos objetos de `ActuadorServo` con distinto pin, y los dos LED son dos objetos de `IndicadorLED`, sin duplicar código.

2. Polimorfismo

El método `ejecutar()` se declara abstracto en `NodoIoT` y cada subclase lo implementa de forma distinta: en `SensorRFID` lee una tarjeta, en `SensorHuella` captura y busca una huella, en `ActuadorServo` abre y cierra la cerradura, en `IndicadorLCD` muestra un mensaje y en `IndicadorLED` parpadea. Como todos los objetos son también `NodoIoT`, el programa puede guardarlos en un arreglo de punteros a la superclase y llamar a `iniciar()` o `ejecutar()` dentro de un ciclo, y cada objeto responde según su clase.

3. Encapsulamiento y ocultamiento de datos

Son **privados** los datos internos de cada componente: `pinSS`, `pinRST`, `ultimoUID`, `idHuella`, `confianza`, `angAbierto`, `angCerrado`, `columnas`, `filas`, `encendido`, `ultimaActividad` y el método `valida()`. Son **protegidos** los que deben ser visibles para las subclases pero no para el resto del programa: `idNodo`, `ubicacion`, `conectado`, `umbral`, `pin` y `dirI2C`. El acceso se hace mediante métodos públicos que mantienen el estado consistente (`abrir()`, `cerrar()`, `encender()`, `mostrar()`, `leerUID()`, `verificar()`): por ejemplo, el programa nunca escribe un ángulo directamente en el servo ni cambia un pin, solo pide la acción. Los pines y ángulos se fijan una vez en el constructor y no se exponen setters para modificarlos, lo que protege el hardware de valores inválidos.

Sección 4: Evidencias de Funcionamiento (Validación Dual)

4.1. Captura de pantalla de la terminal / monitor serial

[INSERTAR AQUÍ LA CAPTURA DE PANTALLA DE LA TERMINAL DE SOFTWARE]

4.2. Fotografía del circuito físico armado (garantía de autoría)

[INSERTAR AQUÍ LA FOTOGRAFÍA DEL CIRCUITO ARMADO]
Debe aparecer un papel escrito a mano con: fecha de la sesión, número de práctica y matrícula de cada integrante.

4.3. Enlace directo al video demostrativo (máximo 1.5 minutos)

Enlace de visualización: [PEGAR AQUÍ EL ENLACE AL VIDEO DEMOSTRATIVO]